RentalPro

Digital Rental Management System

IT402 Capstone Project Defense · All Six Deliverables Complete + Billing Engine + Full Arabic + Production Hardening

Presented by: Abdur-Rahmaan Muhammad Ali

Course: IT402 — IT Capstone II

The Problem

Independent landlords and small property managers still run their business on spreadsheets, paper leases, and manual reminders.

1. **No single source of truth**

   Rent, leases, and maintenance history live in different spreadsheets and email threads.

2. **Missed rent and renewals**

   Nothing proactively flags overdue rent or an expiring lease before it becomes a problem.

3. **No accountability trail**

   Verbal maintenance requests and cash payments leave no audit trail for disputes.

4. **No self-service for tenants**

   Tenants have to call or email for basics: a balance, a receipt, a repair update.

Target Users

- Property Owners
- Property Managers
- Tenants
- Maintenance Staff
- Administrators

“One system, five roles, zero spreadsheets.”

Project Objectives

1	**Centralize property & lease data**	One record per property, unit, tenant, and lease — no duplicate spreadsheets.
2	**Automate the reminders that get missed**	Background jobs alert on lease expiry, rent due, overdue rent, and overdue maintenance.
3	**Give every role a role-scoped view**	RBAC + ownership checks so an Owner never sees another Owner's portfolio.
4	**Make every record auditable**	Immutable payments and an append-only audit log — no silent edits.
5	**Ship it fully tested, not just working**	100% of SRS requirements implemented and covered by an automated test.

Requirements — IT401 Deliverable 1

The Software Requirements Specification defined 25 use cases across six functional areas.

Auth & RBAC

2 use cases

Login, role-based access for 5 roles

Property & Unit Mgmt

4 use cases

Add/edit properties, units, dashboard

Tenant Lifecycle

5 use cases

Onboarding, invites, leases, portal

Rent & Payments

6 use cases

Recording, statements, alerts, history

Maintenance

6 use cases

Submit through close, cost tracking

Reporting

2 use cases

Performance reports, PDF/JSON export

System Design — IT401 Deliverable 2

UML class model and a normalized relational schema — 13 tables, all linked by UUID foreign keys.

`users`	`properties`	`units`	`tenants`
`tenant_invitations`	`password_resets`	`leases`	`rent_payments`
`maintenance_requests`	`maintenance_costs`	`maintenance_notes`	`notifications`
`audit_logs`			

Design decisions carried through to the build

UUID primary keys (identical schema on SQLite and PostgreSQL) • Naive UTC datetimes • Soft-status fields, never physical deletes • Immutable payments • JSON audit diffs

Architecture

A layered Flask application — five layers, each with a single responsibility.

PRESENTATION	40 Jinja2 templates, role-aware top navigation, English/Shona/Arabic menu
APPLICATION	10 Blueprints — auth, dashboard, properties, tenants, rent, maintenance, admin, reports, portal, api
DOMAIN / SERVICE	security.py (RBAC, IDOR), web_security.py (headers, secret key), i18n.py (English/Shona/Arabic), scheduler_jobs.py, services/ (pdf, excel, notifications, payments, storage)
DATA	18 SQLAlchemy models (13 designed + 5 billing), UUID keys, schema managed by Alembic migrations
INFRASTRUCTURE	PostgreSQL/SQLite · Redis · Docker Compose · local or S3-compatible file storage · dev-mode fallback for SendGrid/Twilio/Stripe

Key Features by Role

Owner	Manager	Tenant	Staff	Admin
Create & edit properties/units	Manage assigned properties	Self-service portal	Personal work queue only	User & role management
Configure late fees	Onboard & blacklist tenants	Pay rent online	Update ticket status	Create staff accounts
Full portfolio reports	Assign maintenance work	Submit requests with photos	Attach completion photos	Full data visibility

IT402 Deliverable 2 — 100% Feature Completion

Eleven gaps closed against the SRS after the IT401 Deliverable 3 prototype, grouped here into eight areas — every one covered by an automated test.

Photo uploads

Property (≤10), unit (≤5), and maintenance (≤3) photos — JPEG/PNG validated by magic bytes, not extension

Unit editing

Edit a unit's details and status after it is created — previously units could only be added

Configurable late fees

Fixed or percentage, applied automatically once rent is overdue

Tenant blacklisting

Flag a tenant to block future leases without touching an existing one

Tenant & lease documents

Upload a tenant's ID; download a lease PDF as management or as the tenant

Admin user console

Role & status management with an "at least one active Owner" guard

Staff self-service accounts

Owner/Manager/Admin can create a Maintenance Staff login directly

CSV & Excel exports

A lease's payment history as CSV; a two-sheet .xlsx portfolio workbook

Extended Scope — The Billing Engine

Eight further billing capabilities going beyond the original SRS — 79 additional tests, all passing.

Operational & sundry charges

One-off charges beyond rent — cleaning, call-outs — billed straight to the lease

Electricity metering

A per-property rate plus manual meter readings; consumption bills itself automatically

Pro rata rent

The first month prorates to a daily rate when a tenant moves in mid-month

Variable / escalating rent

A scheduled rent revision takes effect from a chosen date — past periods stay untouched

Automated recurring billing

A daily scheduler job invoices every elapsed period exactly once, with a full paper trail

Account statements by period

Opening balance, in-period activity, closing balance — plus full tenant self-service access

Payment receipts

Every payment now has a real, downloadable PDF receipt, not just a receipt number

Property expenses & net income

Owner-side operating costs feed a true net income estimate, not just rent collected

Extended Scope — Usability & Localisation

Built for Harare: a clearer interface, two more languages and realistic local data — 27 more tests, all passing.

Property map

Click-to-pin coordinates on every property, plus a portfolio map of all of them

Top-bar redesign

The sidebar became a horizontal top bar with a soft blue-to-burgundy gradient

Harare photography

A full-screen Harare skyline login, and a Harare photo behind each section of the app

English, Shona & Arabic

A language menu in the top bar and on login; Arabic reads right to left; the choice is remembered

Place names stay as entered

Only interface text is translated — a test fails if a Harare place name is ever altered

Tenant-friendly unit labels

Tenants see “Avondale Heights · Unit 101”, not an internal unit code

Logged-in redirect fix

Opening the login page while signed in now goes to your dashboard, not a blank page

Harare demo portfolio

5 properties in Avondale, the CBD, Eastlea and Vainona — 16 units and 9 tenants

Extended Scope — Full Arabic & Production Hardening

The final round: every word in Arabic, a stricter security posture and production-ready data handling — 54 more tests, all passing.

Arabic on every screen

Every page, form option and message — over 600 phrases; a test fails if one is missing

Emails & SMS in each user’s language

Sent in the language the user last chose, including by the nightly jobs

Arabic PDFs & spreadsheets

PDF letters joined and right to left, tables mirrored; Excel sheets open right to left

Currency everywhere

Every amount shown as “USD 1,250.00” — screens, messages, emails and PDFs

Security headers & cookies

Strict Content-Security-Policy with no inline code, no framing, HSTS, HttpOnly cookies

No default secret key

The app refuses to start with a missing or published SECRET_KEY

Database migrations

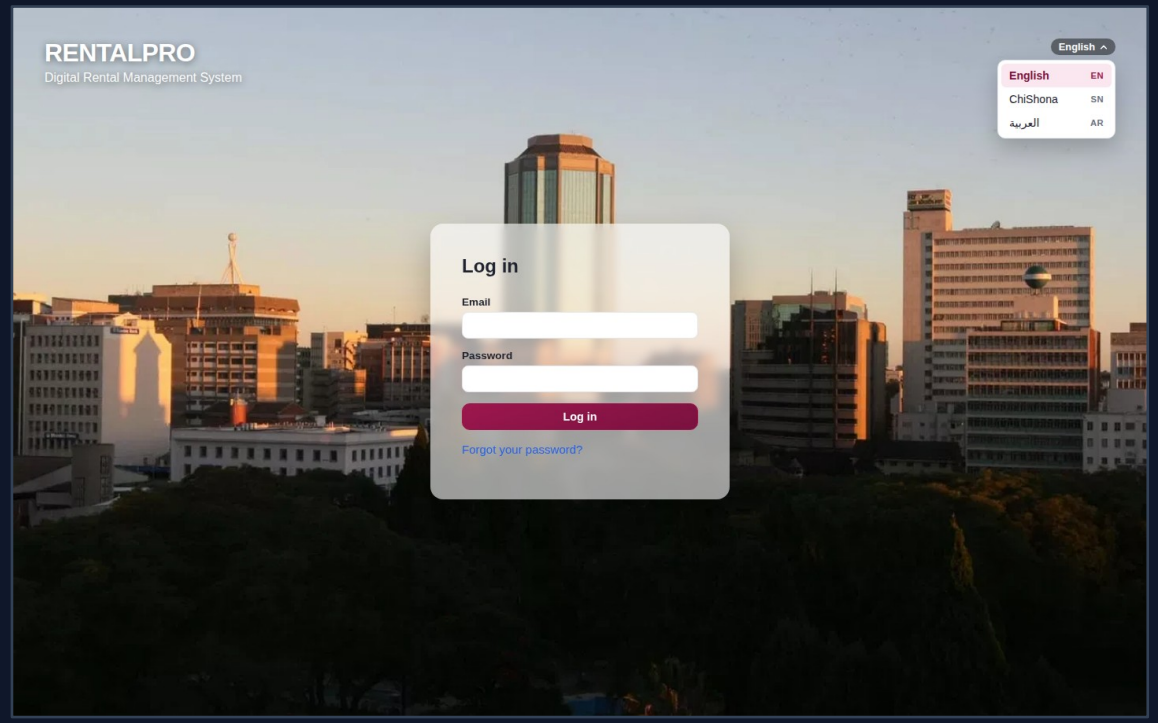
Alembic replaces create_all(); older databases are adopted with their data

Cloud file storage

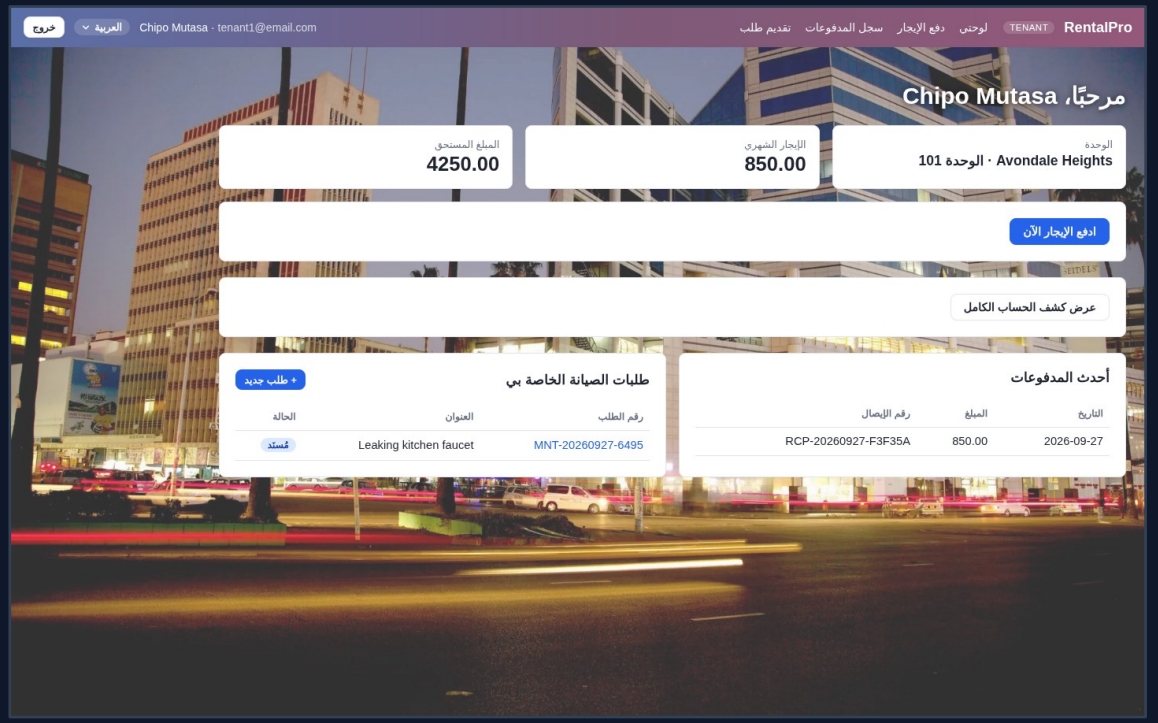
Uploads can live in an S3-compatible bucket and survive redeploys

The New Look — Harare, Shona & Arabic

The Harare login page with its language menu, and the tenant portal in Arabic



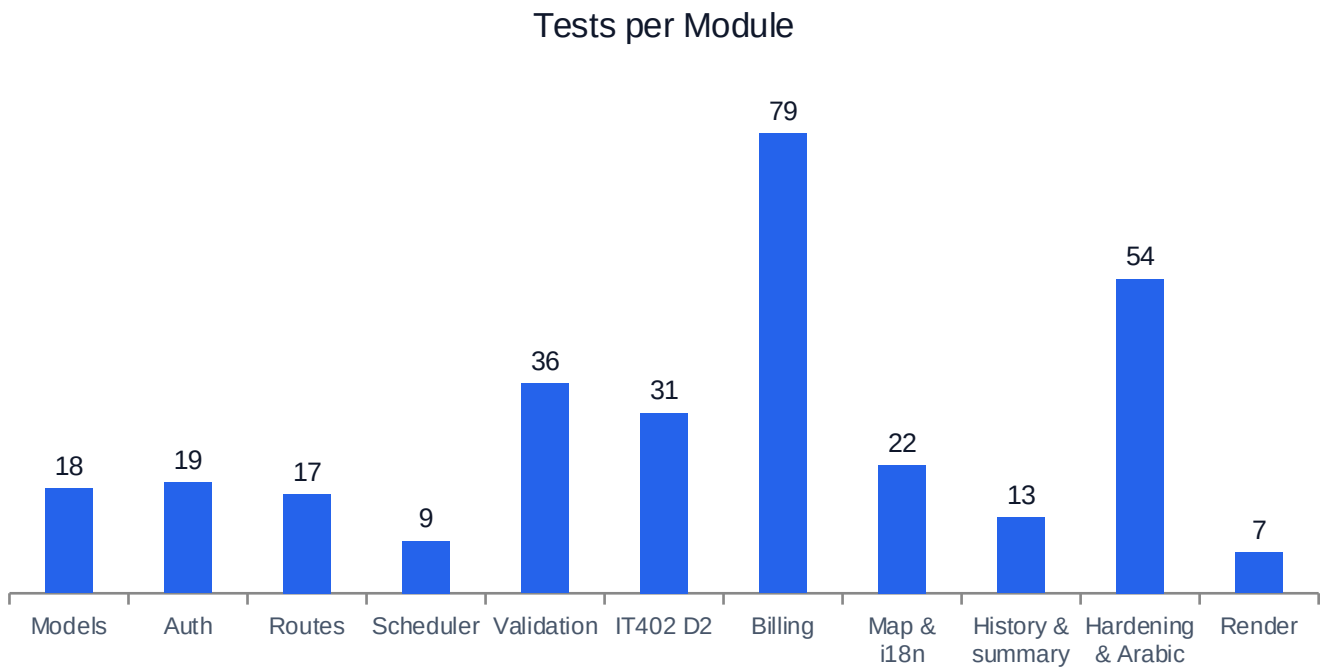
Login
A full-screen Harare skyline, a translucent card, and the language menu open top-right



Tenant portal in Arabic
Laid out right to left — her name and “Avondale Heights” stay exactly as entered

Testing & Quality

305	0	86%	25/25
Automated Tests	Failures	Statement Coverage	Use Cases Covered



What the tests actually verify

- Real HTTP requests through Flask's test client — not isolated function calls
- RBAC across all 5 roles × every management route
- IDOR: a second Owner cannot view the first Owner's property
- Calendar edge cases: mid-month leases, day-31-in-February
- Stripe HMAC signature verification — valid, tampered, expired

Application Demo

The recorded walkthrough of the running application

1. Admin: the Harare dashboard, properties and the portfolio map
2. Rent tracker: a cash payment clears a tenant’s arrears
3. A maintenance ticket, and net income for each property
4. English → Shona → Arabic, right to left, and remembered
5. Tenant portal in Shona; unit history and maintenance summary
6. Manager in Arabic: profile, statement, messages and PDF

Challenges & Solutions

CHALLENGE

Reconciling ambitious design docs with what one developer could actually build and verify

SOLUTION

Simplified IT401 Deliverable 2’s microservice sketch to a layered monolith; corrected IT401 Deliverable 3’s overstated early numbers to the real, verified ones

CHALLENGE

Calendar-accurate billing — a lease starting on the 31st, or a due day that doesn’t exist in February

SOLUTION

Wrote the edge-case tests first (mid-month start, day-31-in-February clipping) before trusting the due-date calculation

CHALLENGE

Testing payments, email, and SMS without any live paid account

SOLUTION

Built a dev-mode fallback pattern into every external service — the whole app runs and tests fully offline

Security Controls

Passwords & secret key

pbkdf2:sha256, 600,000 iterations; no start without a real SECRET_KEY

RBAC

role_required() on every protected route, 5 roles

IDOR protection

assert_owner() / assert_tenant_self() — resource-level, not just route-level

CSRF & cookies

Flask-WTF on every form (signed Stripe webhook exempt); HttpOnly, SameSite session cookie

Headers & lockout

Strict Content-Security-Policy, no framing, HSTS; 5 failed logins lock an account

Upload validation

Magic-byte checks — PDF, JPEG, PNG — regardless of file extension

Audit log

Immutable, append-only record of every state-changing action

Webhook verification

Manual HMAC check, 300-second replay window, independent of the SDK

Conclusion

RentalPro was carried from requirements through design, a working prototype, a full project report, 100% feature completion, a billing engine, and a fully Arabic, production-hardened Harare interface — every claim backed by a running, tested system.

- ✓ 25 / 25 use cases implemented, each with a route and a covering test
- ✓ 13-table database design implemented exactly as specified
- ✓ 305 automated tests, 0 failures, 86% statement coverage
- ✓ 11 feature gaps closed in IT402 Deliverable 2, a billing engine, and English/Shona/Arabic across every screen and document
- ✓ Runs fully offline — zero paid external accounts required to build or test

> *“A claim in this project should be a fact about the running system, not a target for it.”*

Future Work

Beyond IT402:

1	**Native-speaker review**	Have Arabic and Shona speakers check the wording, and extend Shona beyond the main screens
2	**Mobile-optimized tenant portal**	Tenants are the role most likely to use the system from a phone
3	**Live production use**	render.yaml deploys to Render; real tenants still need S3 uploads and a long-term database plan
4	**Live third-party verification**	End-to-end tests against real Stripe, SendGrid, and Twilio sandbox accounts
5	**Two-factor sign-in**	A second factor for Owner, Manager and Admin accounts

Thank You

Questions?